Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata (DIEM)

Documentazione del progetto

Sviluppo di un sistema di antifurto domestico e intelligente

Corso di Internet of things – Prof. Fotia L. – A.A. 2025/2026

SHS

Gruppo 17 composto da:

- Dana Iannaccone (matr: 0612709501)
- Mara Mariano (matr: 0612709247)
- Vincenzo Galdiero (matr: 0612709367)

Indice

1. Introduzione al progetto

2. Requisiti del sistema

2.1. Requisiti funzionali
2.2. Requisiti non funzionali

3. Architettura generale

4. Componenti hardware

4.1. Elenco dei componenti hardware utilizzati
4.2. Componenti con approfondimento tecnico e datasheet
4.2.1. Accelerometro
4.2.2. Sensore PIR
4.2.3. Sensore magnetico
4.2.4. Tastierino numerico

5. Componenti software

5.1. Classe BUZZER
5.2. Classe LED
5.3. Classe OLEDDisplay
5.4. Classe PIR
5.5. Classe Button
5.6. Classe SensoreMagnetico
5.7. Classe StepMotor
5.8. Classe Tastierino
5.9. Classe Accelerometro
5.10. Considerazioni conclusive sui componenti software

6. Comunicazione IoT

7. Appendice

Capitolo 1
Introduzione al progetto

Il presente progetto, realizzato nell'ambito del corso di *Internet of Things*, ha come obiettivo la progettazione e l'implementazione di un sistema di antifurto domestico intelligente.

Il sistema di sicurezza deve essere capace di rilevare eventi di intrusione all'interno di un'abitazione e di reagire in modo automatico e tempestivo, offrendo al contempo all'utente strumenti di controllo locale e remoto. Inoltre il sistema proposto integra componenti hardware e software, consentendo la comunicazione dei dati in rete, la tracciabilità degli eventi e l'analisi del comportamento dell'impianto nel tempo.

Il cuore del sistema è costituito da una scheda ESP32, scelta per le sue caratteristiche di versatilità, potenza di calcolo e connettività WiFi integrata. L'ESP32 svolge il ruolo di nodo centrale dell'architettura IoT, occupandosi della gestione dei sensori, degli attuatori e della logica di funzionamento dell'antifurto. Il software di controllo è stato sviluppato in MicroPython, linguaggio che consente una programmazione chiara e modulare, particolarmente adatta allo sviluppo su microcontrollori.

Un elemento distintivo del progetto è l'adozione di un'architettura distribuita, in cui i dati generati dal sistema vengono trasmessi tramite protocollo MQTT a una piattaforma di elaborazione esterna. Attraverso l'integrazione con Node-RED, è possibile visualizzare e modificare lo stato dell'impianto, archiviare gli eventi rilevati e generare statistiche utili all'analisi del funzionamento del sistema. Inoltre, l'integrazione con un bot Telegram consente all'utente di interagire con l'antifurto da remoto, permettendo l'inserimento e il disinserimento dell'allarme, la chiusura e l'apertura della tenda e la ricezione di notifiche in tempo reale in caso di eventi critici.

Il progetto è stato concepito con l'obiettivo di applicare in modo concreto i concetti teorici affrontati durante il corso. La realizzazione dell'antifurto domestico intelligente rappresenta quindi un caso di studio completo, che mette in evidenza le potenzialità delle tecnologie IoT nell'ambito della sicurezza domestica.

Capitolo 2
Requisiti del sistema

La definizione dei requisiti consente di descrivere in modo formale le funzionalità e le caratteristiche del sistema di antifurto sviluppato. I requisiti sono stati individuati analizzando il comportamento complessivo del sistema e le modalità di interazione previste per l'utente. Essi sono suddivisi in requisiti funzionali, relativi alle operazioni svolte dal sistema, e requisiti non funzionali, relativi alle sue proprietà qualitative.

2.1 Requisiti funzionali

RF1 – Gestione dell'accesso tramite tastierino

Il sistema deve permettere all'utente di inserire e disinserire l'antifurto mediante un tastierino numerico, attraverso la digitazione di un codice di accesso.

RF2 – Selezione della modalità di funzionamento

Il sistema deve consentire la selezione della modalità operativa, permettendo all'utente di scegliere tra modalità giorno e modalità notte.

RF3 – Attivazione ritardata dell'allarme in modalità giorno

In modalità giorno, il sistema deve prevedere un intervallo temporale che consenta all'utente di uscire dall'abitazione prima dell'attivazione dell'allarme.

RF4 – Gestione del tempo di ingresso

In presenza di un evento rilevato in modalità giorno, il sistema deve consentire un intervallo temporale entro il quale l'utente può disinserire l'allarme inserendo il codice corretto.

RF5 – Attivazione immediata in modalità notte

In modalità notte, il sistema deve attivare immediatamente l'allarme al rilevamento di eventi sospetti, senza prevedere tempi di ingresso.

RF6 – Rilevamento di apertura di porta e finestra

Il sistema deve rilevare l'apertura di porta e finestra mediante sensori perimetrali dedicati.

RF7 – Rilevamento del movimento

Il sistema deve essere in grado di rilevare la presenza di movimenti all'interno dell'ambiente, in base alla modalità operativa selezionata.

RF8 – Controllo di dispositivi di segnalazione

In caso di intrusione, il sistema deve attivare i dispositivi di segnalazione previsti per avvisare della presenza di un evento critico.

RF9 – Trasmissione degli eventi

Il sistema deve inviare gli eventi rilevati a un sistema esterno tramite protocollo MQTT, includendo informazioni essenziali per la loro identificazione.

RF10 – Gestione remota del sistema

Il sistema deve consentire il controllo remoto dell'antifurto, permettendo l'inserimento e il disinserimento dell'allarme tramite interfacce software dedicate.

RF11 – Notifiche all'utente

Il sistema deve notificare l'utente in tempo reale in caso di eventi di intrusione, tramite un servizio di messaggistica.

2.2 Requisiti non funzionali

RNF1 – Affidabilità del sistema

Il sistema deve garantire un funzionamento stabile e coerente con la modalità operativa selezionata, riducendo il rischio di attivazioni indesiderate.

RNF2 – Modularità del software

Il software deve essere strutturato in moduli indipendenti, in modo da facilitare la manutenzione del sistema.

RNF3 – Tracciabilità degli eventi

Il sistema deve consentire la memorizzazione degli eventi rilevati, permettendo la consultazione e l'analisi dei dati nel tempo.

RNF4 – Usabilità

Il sistema deve offrire un'interazione semplice e intuitiva, risultando facilmente utilizzabile anche da utenti non esperti.

Capitolo 3

Architettura generale

L'architettura del sistema di antifurto è stata progettata sulla separazione tra livello di acquisizione dei dati, livello di controllo e livello applicativo.

Il livello di **acquisizione e controllo** è costituito dalla scheda ESP32, che rappresenta il nodo centrale del sistema. A essa sono collegati i sensori e gli attuatori utilizzati per il funzionamento dell'antifurto. L'ESP32 si occupa della gestione della logica di controllo, dell'elaborazione degli eventi rilevati e dell'attivazione delle azioni previste in base alla modalità operativa selezionata. Il software eseguito sulla scheda è responsabile della coordinazione dei componenti hardware e della gestione degli stati del sistema.

Il livello di **comunicazione** è realizzato tramite il protocollo MQTT, utilizzato per la trasmissione degli eventi generati dal sistema verso componenti esterni. L'adozione di un protocollo di tipo publish/subscribe consente una comunicazione leggera ed efficiente, particolarmente adatta a sistemi IoT.

Il livello **applicativo** è rappresentato dalla piattaforma Node-RED, che riceve gli eventi inviati dal dispositivo IoT e ne consente la gestione, la categorizzazione e l'archiviazione. Attraverso Node-RED è possibile visualizzare lo stato del sistema, analizzare i dati raccolti e fornire un'interfaccia grafica per il monitoraggio e il controllo dell'antifurto. Questo livello permette inoltre l'elaborazione degli eventi a fini statistici, offrendo una visione complessiva del comportamento dell'impianto nel tempo.

All'interno dell'architettura è prevista anche l'integrazione con un servizio di messaggistica (Telegram), che consente l'interazione remota con il sistema. Tale integrazione permette all'utente di controllare l'antifurto a distanza e di ricevere notifiche in tempo reale in caso di eventi critici, aumentando il livello di accessibilità e sicurezza del sistema.

La suddivisione dell'architettura in livelli distinti consente di isolare le diverse responsabilità del sistema, semplificando la manutenzione .

Capitolo 4

Componenti hardware

Il sistema di antifurto domestico intelligente è realizzato mediante l'integrazione di diversi componenti hardware, comprendenti sensori, attuatori e dispositivi di interazione con l'utente. La scelta dei componenti è stata effettuata in funzione delle funzionalità richieste dal sistema e della loro compatibilità con la piattaforma ESP32.

I componenti hardware utilizzati possono essere suddivisi in **sensori di input**, responsabili della rilevazione degli eventi, e **attuatori e dispositivi di output**, impiegati per la segnalazione e l'interazione con l'utente.

4.1 Elenco dei componenti hardware utilizzati

I principali componenti hardware impiegati nel progetto sono i seguenti:

- **Accelerometro MPU-6050**, utilizzato per il rilevamento di variazioni di movimento o inclinazione sulla finestra;
- **Sensore PIR HC-SR501**, impiegato per il rilevamento del movimento all'interno dell'ambiente;
- **Sensore magnetico KY-003**, utilizzato per il rilevamento dell'apertura della porta;
- **Tastierino numerico**, utilizzato per l'inserimento del codice e la selezione della modalità di funzionamento;
- **Buzzer**, utilizzato come dispositivo di segnalazione acustica;
- **LED**, utilizzato come indicatore visivo dello stato del sistema;
- **Display OLED SSD1306**, utilizzato per la visualizzazione di informazioni e messaggi di stato;
- **Stepper motor 28BYJ-48 con driver ULN2003**, utilizzato per il controllo della tenda motorizzata;
- **Bottone di reset**, utilizzato per il ripristino controllato del sistema.

4.2 Componenti con approfondimento tecnico e datasheet

Alcuni sensori utilizzati nel progetto non sono stati affrontati durante le attività di laboratorio. Per tali componenti è stato quindi effettuato uno studio specifico tramite consultazione dei relativi datasheet, al fine di comprenderne le caratteristiche tecniche e le modalità di utilizzo.

4.2.1 Accelerometro

L'accelerometro è un sensore utilizzato per la misura dell'accelerazione e delle variazioni di movimento lungo più assi. Nel sistema di antifurto, il sensore è impiegato per il rilevamento di movimenti anomali o tentativi di manomissione della struttura.

Dall'analisi del datasheet è stato possibile individuare le principali caratteristiche tecniche del sensore, fondamentali per una corretta integrazione nel sistema, come riportato in tabella.

Parametri principali dell'accelerometro

Parametro	Valore
Numero di assi	3
Range di misura	±2g / ±4g / ±8g
Interfaccia di comunicazione	I²C
Tensione di alimentazione	3.3 V
Frequenza di aggiornamento	Configurabile

Il datasheet completo dell'accelerometro è riportato in **Appendice A**.

4.2.2 Sensore PIR

Il sensore PIR (Passive Infrared) consente il rilevamento del movimento tramite la variazione della radiazione infrarossa emessa dai corpi presenti nell'ambiente. All'interno del sistema di antifurto, il sensore PIR è utilizzato per il rilevamento di intrusioni durante la modalità giorno.

Lo studio del datasheet ha permesso di analizzare i parametri operativi principali del sensore, utili per il corretto posizionamento e per la riduzione dei falsi allarmi.

Parametri principali del sensore PIR

Parametro	Valore
Angolo di rilevamento	~120°
Distanza di rilevamento	fino a 7 m
Tensione di alimentazione	5 V
Tempo di risposta	Regolabile
Tipo di uscita	Digitale

Il datasheet completo del sensore PIR è riportato in **Appendice B**.

4.2.3 Sensore magnetico

Il sensore magnetico è impiegato per la rilevazione dell'apertura e della chiusura della porta d'ingresso. Il funzionamento è basato su un contatto reed che varia il proprio stato in presenza o assenza del magnete.

Attraverso l'analisi del datasheet sono state individuate le caratteristiche elettriche e meccaniche rilevanti per l'utilizzo nel sistema di antifurto.

Parametri principali del sensore magnetico

Parametro	Valore
Tipo di contatto	Reed
Stato di uscita	Aperto / Chiuso
Tensione di funzionamento	3.3 V
Modalità di collegamento	Digitale
Applicazione	Porte e finestre

Il datasheet completo del sensore magnetico è riportato in **Appendice C**.

Tastierino numerico

Il tastierino numerico rappresenta il principale dispositivo di interazione locale con l'utente. È utilizzato per l'inserimento del codice di accesso e per la selezione della modalità operativa del sistema.

Dallo studio del datasheet è emersa la struttura a matrice del tastierino, che richiede una gestione specifica delle linee di riga e colonna per la corretta rilevazione dei tasti premuti.

Parametri principali del tastierino numerico

Parametro	Valore
Struttura	Matrice 4×4
Numero di tasti	16

Tipo di segnale	Digitale
Metodo di scansione	Riga/Colonna
Tensione di funzionamento	3.3 V

Il datasheet completo del tastierino numerico è riportato in **Appendice D**.

Capitolo 5
Componenti software

Il software del sistema di antifurto è organizzato secondo un approccio modulare, basato sulla definizione di una classe per ciascun sensore e attuatore utilizzato. Ogni classe incapsula il comportamento del componente fisico corrispondente, offrendo un'interfaccia chiara al resto del sistema. In questo capitolo vengono descritte le principali classi sviluppate e i metodi più significativi che ne caratterizzano il funzionamento.

5.1 Classe BUZZER

La classe BUZZER gestisce il cicalino collegato alla scheda ESP32 tramite un'uscita PWM. Il costruttore inizializza il segnale PWM sul pin specificato e imposta il duty cycle a zero, in modo che il buzzer risulti spento all'avvio. Oltre alla configurazione hardware, la classe mantiene alcune variabili interne che le permettono di realizzare pattern sonori diversi in modo non bloccante.

Metodi principali

- **alarm_porta()**
  Questo metodo genera un allarme sonoro "modulato" associato, ad esempio, all'apertura della porta. Alla prima chiamata, il metodo inizializza lo stato interno (*flag di attivazione, tempo dell'ultimo aggiornamento, passo temporale, duty cycle*); alle chiamate successive, varia la frequenza del segnale PWM nel tempo utilizzando una funzione sinusoidale. In questo modo il buzzer produce un suono continuo la cui tonalità oscilla, risultando più evidente rispetto a un tono fisso.

- **alarm_finestra()**
  Questo metodo realizza un allarme di tipo intermittente, pensato per segnalare eventi come l'apertura della finestra. Il funzionamento è basato su una logica a stati: il buzzer viene acceso e spento alternativamente, con una pausa prefissata tra un impulso e l'altro. Il numero di accensioni è configurabile tramite il parametro di ripetizione. Quando il conteggio delle ripetizioni raggiunge il valore stabilito, il metodo disattiva l'allarme e spegne il cicalino.

- **play()**
  Il metodo play() consente di riprodurre una breve "melodia", rappresentata come una sequenza di frequenze. Per ogni nota, il metodo imposta la frequenza PWM corrispondente e un duty cycle definito; nel caso di una nota

pari a zero, il buzzer viene temporaneamente spento. Al termine della sequenza, viene richiamato il metodo di arresto.

- **stop()**
  Il metodo stop() ha il compito di riportare il buzzer in uno stato neutro: disattiva eventuali pattern sonori in corso e azzera il duty cycle, assicurando lo spegnimento del dispositivo.

5.2 Classe LED

La classe LED incapsula la gestione di un semplice indicatore luminoso collegato a un pin digitale della ESP32. Il costruttore riceve il numero di pin e un parametro che indica se il LED è attivo in logica alta o bassa, permettendo di adattarsi a diverse configurazioni hardware.

Metodi principali

- **on() e off()**
  I due metodi permettono rispettivamente di accendere e spegnere il LED, impostando il livello logico opportuno sul pin in uscita.

- **blink_once()**
  Questo metodo realizza un singolo lampeggio del LED, attivandolo per un intervallo di tempo prefissato e poi spegnendolo. L'implementazione è di tipo bloccante, in quanto il metodo utilizza una pausa temporale (sleep) per mantenere il LED acceso e poi spento per la durata specificata.

- **blink()**
  Il metodo blink() implementa un lampeggio non bloccante. Sfruttando il tempo di sistema, confronta l'istante corrente con l'ultimo cambio di stato e, se è trascorso l'intervallo desiderato, inverte lo stato del LED. In questo modo il lampeggio può avvenire in parallelo con il resto della logica del sistema, senza interrompere l'esecuzione del programma principale.

5.3 Classe OLEDDisplay

La classe OLEDDisplay gestisce il display OLED collegato tramite interfaccia I²C. Il costruttore inizializza il bus I²C, crea l'oggetto display utilizzando il driver dedicato e pulisce lo schermo, garantendo una situazione iniziale nota.

La classe fornisce sia metodi generici, sia una serie di schermate predefinite che corrispondono agli stati principali dell'antifurto.

Metodi generici

- **clear()**
  Pulisce completamente il contenuto del display e aggiorna la visualizzazione.

- **show_text()**
  Permette di mostrare una stringa di testo in una posizione specificata dallo sviluppatore, ideale per messaggi semplici o temporanei.

- **show_logo()**
  Visualizza un'immagine (logo) caricata sotto forma di buffer grafico, utilizzando le funzionalità di framebuf.

Schermate personalizzate

La classe include una serie di metodi dedicati a specifiche schermate dell'interfaccia utente, tra cui:

- **standby_scr()**: mostra lo stato di sistema disinserito e invita l'utente a inserire il codice.

- **show_code_scr()**: visualizza la maschera per l'inserimento del codice, sostituendo le cifre con asterischi.

- **porta_aperta_scr()**: segnala sul display l'apertura della porta.

- **countdown_scr() e uscita_scr()**: gestiscono graficamente i tempi di ingresso e uscita, mostrando sia il tempo residuo sia una barra di avanzamento.

- **bentornato_scr(), notte_disattivata_scr() e notte_attivata_scr()**: mostrano messaggi di conferma in seguito al corretto disinserimento dell'allarme.

- **tenda_chiusura_scr()** e **tenda_apertura_scr():** informano l'utente sullo stato di movimento della tenda motorizzata.

- **allarme_scr()**: visualizza la schermata di allarme attivo, invitando l'utente a inserire il codice per disattivarlo.

- **scelta_modalita_scr()**: permette di mostrare sul display le opzioni di selezione tra modalità notte e modalità giorno.

Questi metodi consentono di separare la logica applicativa dalla gestione dell'interfaccia utente, rendendo il codice più leggibile e facilmente estendibile.

5.4 Classe PIR

La classe PIR rappresenta il sensore di movimento a infrarossi passivo. Il costruttore crea un oggetto Pin configurato come ingresso digitale, collegato al sensore.

Metodi principali

- **motion_detected()**
  Il metodo restituisce un valore booleano che indica la presenza o meno di movimento. Internamente verifica il valore logico letto dal pin, consentendo alla logica dell'antifurto di reagire in caso di rilevazione di movimento.

5.5 Classe Button

La classe Button è stata progettata per gestire in modo sicuro e affidabile il pulsante di riavvio del sistema antifurto, evitando l'uso di interrupt hardware e consentendo l'esecuzione di logica applicativa prima del reset del microcontrollore.

Il pulsante è collegato a un pin GPIO configurato come ingresso con resistenza di pull-up interna (PULL_UP). In questa configurazione il pin assume valore logico alto (1) in condizioni di riposo e valore logico basso (0) quando il pulsante viene premuto.

Metodi principali

- **is_pressed (self):**
  Il metodo consente di rilevare in modo affidabile la pressione del pulsante di reset applicando un semplice debounce software. In una configurazione con pull-up interno, il pin assume valore logico basso quando il pulsante viene premuto. Per evitare false attivazioni dovute al rimbalzo meccanico del contatto, il metodo introduce un breve ritardo e verifica nuovamente lo stato del pin. Solo se la condizione di pressione risulta stabile viene restituito True, garantendo una lettura corretta e sicura del pulsante.

5.6 Classe SensoreMagnetico

La classe SensoreMagnetico modella il sensore di apertura utilizzato, ad esempio, su porte o finestre. Il costruttore configura il pin in ingresso con resistenza di pull-up, in modo che il valore logico cambi in funzione della vicinanza o meno del magnete.

Metodi principali

- **is_open()**
  Il metodo restituisce un valore booleano che indica se il contatto risulta aperto. In pratica, permette di sapere se la porta o la finestra associata al

sensore è in stato di apertura, fornendo un'informazione fondamentale per la logica di allarme.

5.7 Classe StepMotor

La classe StepMotor gestisce un motore passo-passo a quattro fili, utilizzato, ad esempio, per il movimento della tenda. Il costruttore riceve i pin dedicati alle bobine del motore, il ritardo tra un passo e l'altro e il numero di passi per giro, e inizializza la sequenza di eccitazione delle bobine. All'avvio, viene richiamato un metodo di rilascio per disattivare le bobine e ridurre i consumi.

Metodi principali

- **_write()**
  Metodo interno che imposta i livelli logici sui pin del motore in base al pattern specificato, controllando quali bobine devono essere attivate a ogni passo.

- **stop()**
  Imposta un flag interno che consente di interrompere in modo ordinato il movimento del motore durante l'esecuzione di una rotazione.

- **release()**
  Disattiva tutte le bobine del motore, portando a zero i segnali sui pin. È utile sia a fine movimento, sia per ridurre riscaldamento e consumi quando il motore non è in uso.

- **move_steps()**
  Esegue un numero di passi specificato in una determinata direzione (oraria o antioraria). Il metodo aggiorna l'indice della sequenza di eccitazione, scrive il nuovo pattern sulle bobine e inserisce una breve pausa tra un passo e l'altro. Il flag di stop, se impostato, permette di interrompere prematuramente il movimento.

- **move_turns()**
  Calcola il numero di passi corrispondente a un certo numero di giri e richiama move_steps(). In questo modo è possibile comandare il motore in termini di giri completi anziché singoli passi.

5.8 Classe Tastierino

La classe Tastierino gestisce un tastierino numerico organizzato a matrice di righe e colonne. Il costruttore riceve i pin fisici delle righe e delle colonne, oltre alla mappa

dei tasti (matrice di caratteri), e configura i pin corrispondenti: le righe come ingressi con pull-up e le colonne come uscite.

Metodi principali

- **scan_key()**
  Il metodo implementa la scansione della matrice. A turno, ciascuna colonna viene portata a livello basso, mentre le righe vengono lette per verificare se un tasto è stato premuto. In caso di pressione, il metodo individua la coppia riga-colonna corrispondente, ricava il simbolo associato dalla mappa dei tasti e attende il rilascio del tasto prima di restituire il valore. Se nessun tasto risulta premuto, il metodo restituisce None.
  Questa logica consente una lettura affidabile del tastierino, gestendo anche il problema dei rimbalzi tramite un breve ritardo e l'attesa del rilascio.

5.9 Classe Accelerometro

La classe Accelerometro gestisce un sensore di tipo MPU collegato via I²C, utilizzato per il rilevamento delle accelerazioni lungo i tre assi. Il costruttore inizializza il bus I²C, imposta l'indirizzo del dispositivo e provvede alla configurazione di base del sensore, disattivando lo stato di sleep tramite la scrittura su un registro di controllo. Inoltre, viene inizializzata una soglia di sensibilità, utilizzata per determinare condizioni di allarme.

Metodi principali

- **set_soglia()**
  Questo metodo permette di aggiornare dinamicamente la soglia di sensibilità, ad esempio in seguito a un comando proveniente da Node-RED. Il valore ricevuto viene convertito in numero e, in caso di successo, sostituisce la soglia precedente. In caso di errore, viene generato un messaggio di diagnostica.

- **_read_raw_data()**
  Metodo interno che legge due registri consecutivi del sensore (parte alta e bassa del valore), ricostruendo il dato grezzo a 16 bit e gestendo correttamente i valori negativi tramite conversione in complemento a due.

- **read_acceleration()**
  Utilizzando il metodo interno di lettura, il metodo acquisisce i valori grezzi di accelerazione sui tre assi, li scala secondo il fattore previsto dal sensore e restituisce un dizionario contenente le componenti di accelerazione lungo gli assi x, y e z.

- **is_alarm()**
  Questo metodo implementa la logica di rilevazione dell'allarme legato al movimento. Dopo aver letto le accelerazioni sui tre assi, verifica se il valore assoluto di una delle componenti supera la soglia impostata. In tal caso, restituisce True, indicando una condizione potenzialmente anomala; in caso contrario, restituisce False.
  In questo modo l'accelerometro può essere utilizzato come sensore di manomissione o urto, contribuendo ad aumentare il livello di sicurezza del sistema.

5.10 Considerazioni conclusive sui componenti software

Le classi descritte costituiscono la base software per la gestione dei sensori e degli attuatori del sistema di antifurto. Ciascuna di esse incapsula le operazioni essenziali legate al componente fisico corrispondente, offrendo un'interfaccia chiara e riutilizzabile.
Oltre alle classi presentate, il sistema include file di avvio e di configurazione (ad esempio i moduli di boot, configurazione, wifi e logica principale), che coordinano l'utilizzo congiunto delle classi all'interno del ciclo di esecuzione dell'antifurto.

Capitolo 6

Comunicazione IoT

La comunicazione IoT del sistema di antifurto è realizzata mediante il protocollo MQTT. Il sistema utilizza un'architettura publish/subscribe, che consente una comunicazione asincrona e disaccoppiata tra i vari componenti del sistema. Sono presenti tre attori principali:

- **ESP32**: pubblica lo stato del sistema, la telemetria e riceve comandi remoti;
- **Mosquitto**: broker MQTT che gestisce la distribuzione dei messaggi;
- **Node-RED**: piattaforma di elaborazione che si sottoscrive ai topic MQTT dell'ESP32, gestisce la logica applicativa e l'integrazione con Telegram.

6.1 Broker MQTT - Mosquitto

Il broker MQTT utilizzato è Eclipse Mosquitto. Mosquitto si occupa di ricevere i messaggi pubblicati dall'ESP32 e di recapitarli a tutti i client sottoscritti ai topic di interesse, garantendo una comunicazione affidabile e a bassa latenza. La scheda ESP32 si connette al broker tramite rete WiFi e agisce sia come publisher, sia come subscriber. I topic impiegati sono:

- **casa/caveau/status**: all'avvio, il sistema pubblica il messaggio REBOOT per notificare il riavvio del modulo. In caso di evento critico, pubblica stringhe relative allo stato di allarme (ALLARME_PORTA o ALLARME_FINESTRA). Durante il normale funzionamento, trasmette lo stato corrente.
- **casa/caveau/cmd**: è il canale attraverso il quale Node-Red invia istruzioni operative per calibrazione di soglia, controllo della tenda, attivazione modalità di sicurezza.
- **casa/caveau/telemetry/wifi**: invia ogni 2 secondi il valore RSSI (potenza del segnale) della rete WiFi per il monitoraggio della qualità del collegamento.
- **casa/caveau/telemetry/tenda**: fornisce un feedback sullo stato del motore passo-passo.

6.2 Node-RED

Node-RED è utilizzato come dashboard di supervisione e come livello applicativo del sistema IoT. Attraverso nodi MQTT, Node-RED si sottoscrive ai topic pubblicati dall'ESP32 e riceve in tempo reale le informazioni relative allo stato del sistema di antifurto e ai dati di telemetria. In base allo stato del sistema e agli eventi rilevati, Node-RED genera notifiche verso l'utente o pubblica comandi MQTT diretti all'ESP32.

La dashboard è composta da due tabs: "Security Hub" e "Log & Grafici".

Security Hub è a sua volta composta dai group *Pannello Comandi*, *Telemetria*, *Gestione Tenda* e *Calibrazione*.

- **Pannello Comandi** è il nucleo interattivo che permette di visualizzare lo Stato Sistema, tramite un nodo *ui_text*, e di commutare tra le modalità di sicurezza. I comandi vengono inviati all'ESP32 tramite il topic *casa/caveau/cmd* e notificati simultaneamente via Telegram. Inoltre, include un selettore (nodo *ui_doprdown*) per consentire all'utente di classificare manualmente la criticità degli eventi rilevati.
- **Telemetria** è dedicato al monitoraggio delle prestazioni di rete. Consente la visualizzazione della potenza del segnale WiFi (dBm) sia attraverso un grafico temporale (*ui_chart*) che un indicatore a lancetta (*ui_gauge*), permettendo di individuare eventuali instabilità della connessione.
- **Gestione Tenda** offre la possibilità di inviare comandi di apertura (*OPEN)* e chiusura (*CLOSE*) per lo StepMotor presente su una delle finestre. Inoltre, permette di monitorare lo stato tramite un template grafico dinamico (*ui_template)* che mostra icone animate in caso di movimento ("*Apertura/Chiusura in corso*") o icone statiche a operazione conclusa.
- **Calibrazione** garantisce la regolazione della soglia dell'Accelerometro direttamente dalla dashboard. Tramite un selettore numerico (*ui_numeric*), l'utente può impostare il valore limite da 0.5 a 5G; il nuovo valore viene memorizzato globalmente in Node-RED e trasmesso istantamente all'ESP32 via MQTT (topic *casa/caveau/cmd).*

Log & Grafici è a sua volta composto dai group *Criticità e Sensori, Registro Storico* e *Statistiche Modalità.*

- **Registro Storico** consente la consultazione dei log salvati. Attraverso i pulsanti di controllo l'utente può caricare gli ultimi 25 eventi, visualizzandoli in una tabella formattata che include data e ora, tipo di evento, sensore coinvolto e livello di criticità. È presente anche una funzione di pulizia totale del database.
- **Criticità e sensori** fornisce una rappresentazione dei dati statistici salvati. Include un grafico a barre per la distribuzione della criticità degli eventi e un grafico a torta che esplicita quali sensori sono stati attivati più frequentemente.
- **Statistiche Modalità** presenta un grafico a barre orizzontali che riassume l'utilizzo del sistema, mostrando quante volte l'antifurto ha scatenato un allarme nelle due modalità.

6.3 Bot Telegram

Per l'interazione con l'utente è stato integrato un bot Telegram, gestito tramite Node-RED. Il bot fornisce un'interfaccia semplice e immediata per il monitoraggio e il controllo del sistema di antifurto.

In particolare, il bot Telegram consente:

- l'invio di notifiche automatiche in caso di allarme, specificando sensore coinvolto e modalità del sistema
- l'invio di comandi testuali da parte dell'utente per il controllo remoto dell'antifurto: *total* per attivare la modalità totale, *night* per la modalità notte, *disarmed* per la disattivazione dell'allarme
- il controllo remoto della tenda della finestra, consentito esclusivamente quando il sistema è disarmato, tramite i comandi *open* e *close*.

Il sistema è implementato mediante una logica di validazione dei comandi: solo i messaggi scritti interamenti in carattere minuscolo vengono considerati validi e processati. Questa scelta progettuale riduce il rischio di eventuali interpretazioni ambigue e aumenta l'affidabilità del controllo remoto.

7. Appendice

Appendice A

Datasheet accelerometro:
https://docs.wokwi.com/parts/wokwi-mpu6050

Appendice B

Datasheet sensore PIR:
https://www.mpja.com/download/31227sc.pdf

Appendice C

Datasheet sensore magnetico:
https://www.gotronic.fr/pj2-sen-ky003-manual-1932.pdf?srsltid=AfmBOopfeCe8kKJxU5r5q_h1XKJEEBeENz47PR_s04IjewlWjLjisA7E

Appendice D

Datasheet tastierino numerico:
https://docs.wokwi.com/parts/wokwi-membrane-keypad